

Database Concurrency - Practical Examples

Setup: Sample Table

For all examples, we'll use this simple accounts table:

```
CREATE TABLE accounts (  
  account_id SERIAL PRIMARY KEY,  
  account_name VARCHAR(100),  
  balance DECIMAL(10, 2)  
);  
  
INSERT INTO accounts (account_name, balance) VALUES  
  ('Alice', 1000.00),  
  ('Bob', 500.00),  
  ('Charlie', 750.00);
```

Example 1: Lost Update Problem

This example demonstrates how concurrent updates can result in lost data.

Terminal 1 (Transaction A):

```
BEGIN;  
SELECT balance FROM accounts WHERE account_name = 'Alice';  
-- Returns: 1000.00
```

Terminal 2 (Transaction B):

```
BEGIN;  
SELECT balance FROM accounts WHERE account_name = 'Alice';  
-- Returns: 1000.00
```

Terminal 1 (Transaction A):

```
-- Add $200 deposit  
UPDATE accounts SET balance = 1200.00 WHERE account_name = 'Alice';  
COMMIT;
```

Terminal 2 (Transaction B):

```
-- Add $300 deposit
UPDATE accounts SET balance = 1300.00 WHERE account_name = 'Alice';
COMMIT;
```

Result: Final balance is \$1300, but it should be \$1500! The first update was lost.

Solution: Use Row-Level Locking

```
-- Terminal 1:
BEGIN;
SELECT balance FROM accounts WHERE account_name = 'Alice' FOR UPDATE;
UPDATE accounts SET balance = balance + 200 WHERE account_name = 'Alice';
COMMIT;

-- Terminal 2:
BEGIN;
SELECT balance FROM accounts WHERE account_name = 'Alice' FOR UPDATE;
-- This waits for Terminal 1 to commit
UPDATE accounts SET balance = balance + 300 WHERE account_name = 'Alice';
COMMIT;
```

Result: Final balance is correctly \$1500.

Example 2: Non-Repeatable Read

Demonstrates reading different values within the same transaction.

Terminal 1 (Reader):

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT balance FROM accounts WHERE account_name = 'Bob';
-- Returns: 500.00
```

Terminal 2 (Writer):

```
BEGIN;
UPDATE accounts SET balance = 800.00 WHERE account_name = 'Bob';
COMMIT;
```

Terminal 1 (Reader):

```
SELECT balance FROM accounts WHERE account_name = 'Bob';
-- Returns: 800.00 (different from first read!)
```

```
COMMIT;
```

Solution: Use Repeatable Read Isolation

```
-- Terminal 1:
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE account_name = 'Bob';
-- Returns: 500.00

-- Terminal 2 commits changes here...

SELECT balance FROM accounts WHERE account_name = 'Bob';
-- Still returns: 500.00 (same as first read)
COMMIT;
```

Example 3: Phantom Reads

Demonstrates rows appearing or disappearing between queries.

Terminal 1:

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM accounts WHERE balance > 600;
-- Returns: 2 (Alice: 1000, Charlie: 750)
```

Terminal 2:

```
BEGIN;
INSERT INTO accounts (account_name, balance) VALUES ('Diana', 900.00);
COMMIT;
```

Terminal 1:

```
SELECT COUNT(*) FROM accounts WHERE balance > 600;
-- Returns: 3 (Diana's row is now visible - a "phantom")
COMMIT;
```

Solution: Use Repeatable Read

```
-- Terminal 1:
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
SELECT COUNT(*) FROM accounts WHERE balance > 600;
-- Returns: 2

-- Terminal 2 inserts Diana here...

SELECT COUNT(*) FROM accounts WHERE balance > 600;
-- Still returns: 2 (Diana not visible in this transaction)
COMMIT;
```

Example 4: Deadlock Scenario

Two transactions waiting for each other's locks.

Terminal 1:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_name = 'Alice';
-- Successfully acquires lock on Alice's row
```

Terminal 2:

```
BEGIN;
UPDATE accounts SET balance = balance - 50 WHERE account_name = 'Bob';
-- Successfully acquires lock on Bob's row
```

Terminal 1:

```
UPDATE accounts SET balance = balance + 100 WHERE account_name = 'Bob';
-- Waits for Terminal 2 to release lock on Bob...
```

Terminal 2:

```
UPDATE accounts SET balance = balance + 50 WHERE account_name = 'Alice';
-- Deadlock! PostgreSQL will detect this and abort one transaction
-- ERROR: deadlock detected
```

PostgreSQL Output:

```
ERROR: deadlock detected
DETAIL: Process 12345 waits for ShareLock on transaction 67890;
blocked by process 12346.
```

```
Process 12346 waits for ShareLock on transaction 67891;  
blocked by process 12345.
```

Solution: Consistent Lock Ordering

Always acquire locks in the same order:

```
-- Both transactions update accounts in alphabetical order  
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE account_name = 'Alice';  
UPDATE accounts SET balance = balance + 100 WHERE account_name = 'Bob';  
COMMIT;
```

Example 5: Money Transfer (Common Pattern)

Safely transferring money between accounts.

Unsafe Version:

```
-- Terminal 1:  
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE account_name = 'Alice';  
UPDATE accounts SET balance = balance + 100 WHERE account_name = 'Bob';  
COMMIT;
```

Safe Version with Locking:

```
BEGIN;  
-- Lock both accounts first (in consistent order)  
SELECT balance FROM accounts  
WHERE account_name IN ('Alice', 'Bob')  
ORDER BY account_name  
FOR UPDATE;  
  
-- Verify sufficient funds  
SELECT balance FROM accounts WHERE account_name = 'Alice';  
-- If balance >= 100, proceed:  
  
UPDATE accounts SET balance = balance - 100 WHERE account_name = 'Alice';  
UPDATE accounts SET balance = balance + 100 WHERE account_name = 'Bob';  
COMMIT;
```

Example 6: Using Advisory Locks

PostgreSQL provides application-level locks.

Terminal 1:

```
BEGIN;
SELECT pg_advisory_lock(123456);
-- Acquired lock with ID 123456
SELECT pg_sleep(10); -- Simulate long operation
SELECT pg_advisory_unlock(123456);
COMMIT;
```

Terminal 2:

```
BEGIN;
SELECT pg_advisory_lock(123456);
-- Waits until Terminal 1 releases the lock
SELECT 'Got the lock!';
SELECT pg_advisory_unlock(123456);
COMMIT;
```

Example 7: Monitoring Locks

Check what locks are currently held:

```
-- View all locks
SELECT
    locktype,
    database,
    relation::regclass,
    page,
    tuple,
    virtualxid,
    transactionid,
    mode,
    granted
FROM pg_locks
WHERE pid = pg_backend_pid();

-- View blocking queries
SELECT
    blocked_locks.pid AS blocked_pid,
    blocking_locks.pid AS blocking_pid,
    blocked_activity.query AS blocked_statement,
    blocking_activity.query AS blocking_statement
FROM pg_catalog.pg_locks blocked_locks
JOIN pg_catalog.pg_stat_activity blocked_activity ON blocked_activity.pid
= blocked_locks.pid
JOIN pg_catalog.pg_locks blocking_locks
    ON blocking_locks.locktype = blocked_locks.locktype
    AND blocking_locks.database IS NOT DISTINCT FROM
```

```
blocked_locks.database
    AND blocking_locks.relation IS NOT DISTINCT FROM
blocked_locks.relation
    AND blocking_locks.pid != blocked_locks.pid
JOIN pg_catalog.pg_stat_activity blocking_activity ON
blocking_activity.pid = blocking_locks.pid
WHERE NOT blocked_locks.granted;
```

Example 8: Optimistic Locking Pattern

Using version numbers to detect conflicts.

```
-- Add version column
ALTER TABLE accounts ADD COLUMN version INTEGER DEFAULT 1;

-- Application reads the data
SELECT account_id, balance, version FROM accounts WHERE account_name =
'Alice';
-- Returns: account_id=1, balance=1000, version=1

-- Application updates with version check
UPDATE accounts
SET balance = 900, version = version + 1
WHERE account_name = 'Alice' AND version = 1;

-- If another transaction updated first, this returns 0 rows updated
-- Application must retry the operation
```

Summary

These examples demonstrate:

1. **Lost updates** and how to prevent them with **FOR UPDATE**
2. **Non-repeatable reads** and how isolation levels affect them
3. **Phantom reads** and snapshot isolation
4. **Deadlocks** and prevention through consistent ordering
5. **Safe money transfers** with proper locking
6. **Advisory locks** for application-level coordination
7. **Lock monitoring** to debug concurrency issues
8. **Optimistic locking** for conflict detection

Understanding these patterns is crucial for building reliable concurrent database applications.